

Pet Territory Wars — MVP Domain Model v1.0

Doküman sürümü: 1.0.0

Backend: Go

Mobil: Flutter

Motor: Stateless Game Engine

Kapsam: Faz 0 Backend Simulation + Faz 1 Barebone Mobile

Öncelik: Aynı anda binlerce kullanıcının yürüyüş gönderebildiği ölçeklenebilir yapı

1. Amaç

Bu doküman, Pet Territory Wars MVP'sinde kullanılacak temel domain nesnelerini, sorumluluklarını, ilişkilerini ve aggregate sınırlarını tanımlar.

Domain Model'in amacı doğrudan database tablosu çıkarmak değildir.

Amaç:

- Oyundaki gerçek kavramları tanımlamak
- Game Engine'e verilecek state'i belirlemek
- Kalıcı tutulacak verileri ayırmak
- Hesaplanabilir verileri belirlemek
- Aynı anda yapılan işlemlerde çakışma sınırlarını netleştirmek
- Database ve API tasarımına temel oluşturmak

Temel ayırım:

```
Domain Model
=
İş dünyasındaki kavramlar ve kurallar

Database Model
=
Bu kavramların kalıcı saklama biçimi

API Model
=
Bu kavramların istemciyle iletişim biçimi
```

Bu üç model birbirinin aynısı olmak zorunda değildir.

2. Ölçeklenebilirlik İlkeleri

Domain Model tasarlanırken şu senaryo esas alınacaktır:

Aynı anda binlerce kullanıcı yürüyüş yapabilir.
Yüzlerce kullanıcı aynı mahallede bulunabilir.
Birden fazla kullanıcı aynı hex'e aynı anda saldırabilir.
Harita istemcileri aynı anda binlerce hex okuyabilir.

Bu nedenle model şu prensiplere dayanır:

2.1 Küçük Aggregate Sınırları

Tek bir yürüyüş bütün şehir aggregate'i üzerinden işlenmez.

Yanlış yaklaşım:

City
└ Bütün oyuncular
└ Bütün hex'ler
└ Bütün yürüyüşler

Bu model aynı şehirdeki bütün işlemleri birbirine kilitler.

Doğru yaklaşım:

Walk Session	→ bağımsız işlem
Territory Hex	→ bağımsız rekabet birimi
Player Score	→ bağımsız skor state'i
Season	→ bağımsız dönem bilgisi

2.2 Hotspot Riskini Azaltma

En fazla eşzamanlı güncellenme ihtimali olan nesne:

Territory Hex

Bu nedenle bir hex state'i mümkün olduğunca küçük tutulmalıdır.

Bir hex güncellenirken:

- Oyuncunun bütün profili
- Şehrin bütün leaderboard'u
- Sezonun bütün verisi

aynı transaction içinde kilitlenmemelidir.

2.3 Yazma ve Okuma Modellerini Ayırma

Domain'de authoritative state ile kullanıcıya sunulan hızlı okuma modelleri ayrılmalıdır.

Örnek:

Authoritative:
Territory ownership ve dominance

Read Model:
Harita tile'ları
Mahalle leaderboard'u
Oyuncu profil özeti

Leaderboard gibi yoğun okunan sonuçların her istekte bütün hex'lerden tekrar hesaplanması zorunlu değildir.

Ancak leaderboard read model'i authoritative domain state değildir.

2.4 Eventual Consistency Kullanılabilecek Alanlar

Aşağıdaki verilerin anlık olarak transaction içinde güncellenmesi zorunlu değildir:

- Leaderboard sırası
- Profil istatistik özeti
- Analytics
- Push notification
- Harita cache invalidation
- Lifetime raporlar

Aşağıdaki veriler ise güçlü tutarlılık ister:

- Hex sahibi
- Hex dominance değeri
- Hex version bilgisi
- Yürüyüşün daha önce işlenip işlenmediği
- Ownership transfer sonucu

3. Domain Bounded Context

MVP için tek backend uygulaması kullanılacaktır ancak domain mantıksal olarak şu alanlara ayrılır:

Identity Context
Walk Context
Territory Context

Scoring Context
Season Context
Rules Context
Event Context

Bunlar mikroservis değildir.

Aynı Go uygulaması içinde modüler alanlardır.

4. Temel Domain Nesneleri

MVP için temel domain nesneleri:

Player
Pet
City
Walk
LocationPoint
TerritoryHex
HexChange
PlayerScore
Season
RuleSet
DomainEvent

MVP'de Pet nesnesi minimal tutulacaktır.

Breed bonusu veya pet class sistemi bulunmayacaktır.

5. Player

5.1 Amaç

Oyunu oynayan kullanıcıyı temsil eder.

Player authentication account ile aynı kavram değildir.

Authentication sistemi:

Kimlik doğrulama
Email
Google Sign-In

Apple Sign-In
Token

gibi konularla ilgilenir.

Player domain nesnesi ise oyun kimliğidir.

5.2 Model

```
type Player struct {  
    ID          PlayerID  
    DisplayName string  
    Status      PlayerStatus  
    HomeCityID  CityID  
    CreatedAt   time.Time  
}
```

5.3 PlayerStatus

```
type PlayerStatus string  
  
const (  
    PlayerActive    PlayerStatus = "ACTIVE"  
    PlayerSuspended PlayerStatus = "SUSPENDED"  
    PlayerDeleted   PlayerStatus = "DELETED"  
)
```

5.4 Sorumluluğu

Player şunları temsil eder:

- Oyuncunun oyun kimliği
- Görünen adı
- Oyun durumunu
- Ana şehir ilişkisini

Player şunları doğrudan taşımaz:

- Sahip olduğu bütün hex listesi
- Bütün yürüyüşler
- Leaderboard sırası
- Bütün skor geçmişi
- Raw GPS verisi

Bunların Player içinde tutulması aggregate'i gereksiz büyütür.

6. Pet

6.1 Amaç

Oyuncunun yürüyüş yaptığı evcil hayvanı temsil eder.

MVP'de yalnızca temel profil amacıyla bulunur.

6.2 Model

```
type Pet struct {  
    ID          PetID  
    OwnerID     PlayerID  
    Name        string  
    Status      PetStatus  
    CreatedAt   time.Time  
}
```

MVP'de aşağıdaki özellikler bulunmaz:

Breed bonus
Level
Evolution
Attack power
Defense power
Multiple pet bonus

Bu alanlar ileride eklenebilir ancak Calculator Rules MVP'yi etkilemez.

6.3 İlişki

Player 1 — N Pet

MVP uygulamasında ilk sürümde oyuncu başına yalnızca bir aktif pet sınırı uygulanabilir.

Domain modeli ileride birden fazla pet'i destekleyebilir.

7. City

7.1 Amaç

Oyunun aktif olduğu şehir alanını temsil eder.

7.2 Model

```
type City struct {
    ID          CityID
    Name        string
    CountryCode string
    TimeZone    string
    Status      CityStatus
    BoundaryRef BoundaryReference
    ActiveRuleSet RuleSetVersion
}
```

7.3 CityStatus

```
type CityStatus string

const (
    CityActive      CityStatus = "ACTIVE"
    CityComingSoon CityStatus = "COMING_SOON"
    CityFrozen      CityStatus = "FROZEN"
)
```

7.4 Boundary

Şehir polygon'unun bütün koordinatlarının domain nesnesi içinde taşınması gerekli değildir.

```
type BoundaryReference struct {
    Version string
}
```

Application katmanı motoru çağırırken gerekli boundary verisini hazırlar.

7.5 Ölçek Notu

City aggregate'i bütün hex'leri içermez.

Yanlış:

```
type City struct {
    Hexes []TerritoryHex
}
```

Bu yapı şehir büyüdükçe yönetilemez hale gelir.

Doğru ilişki:

CityID, TerritoryHex üzerinde referans olarak bulunur.

8. Walk

8.1 Amaç

Bir oyuncunun başlattığı ve tamamladığı tek yürüyüş oturumunu temsil eder.

Walk, game engine'in ana işlem birimidir.

8.2 Model

```
type Walk struct {  
    ID                WalkID  
    PlayerID          PlayerID  
    PetID             PetID  
    CityID            CityID  
  
    Status            WalkStatus  
  
    StartedAt         time.Time  
    EndedAt           *time.Time  
    EvaluatedAt       *time.Time  
  
    RuleSetVersion    RuleSetVersion  
    EngineVersion     EngineVersion  
  
    ValidationStatus  WalkValidationStatus  
  
    TotalDistanceM    float64  
    ValidDistanceM    float64  
    DurationSeconds   int  
  
    InputHash         string  
    CreatedAt         time.Time  
}
```

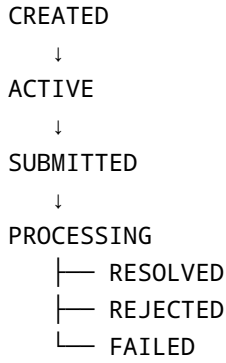
8.3 WalkStatus

```
type WalkStatus string  
  
const (  
    WalkCreated    WalkStatus = "CREATED"  
    WalkActive     WalkStatus = "ACTIVE"
```



```
WalkSubmitted WalkStatus = "SUBMITTED"  
WalkProcessing WalkStatus = "PROCESSING"  
WalkResolved WalkStatus = "RESOLVED"  
WalkRejected WalkStatus = "REJECTED"  
WalkFailed WalkStatus = "FAILED"  
)
```

8.4 State Geçişleri



8.5 Idempotency

Her Walk benzersizdir.

Aynı Walk tekrar gönderilirse iki kez capture üretilmemelidir.

Kimlik:

```
WalkID + PlayerID + InputHash
```

Walk state'i idempotency kontrolünün temelidir.

8.6 Aggregate Sınırı

Walk aggregate'i şunları yönetir:

- Yürüyüş lifecycle
- Input kimliği
- Validation sonucu
- İşlenme durumu
- Kullanılan engine ve RuleSet sürümü

Walk aggregate'i TerritoryHex ownership işlemini kendi içinde yönetmez.

9. LocationPoint ve Route

9.1 Amaç

Flutter uygulamasından gelen GPS örneklerini temsil eder.

9.2 Model

```
type LocationPoint struct {  
    Sequence      int  
    Latitude      float64  
    Longitude     float64  
    RecordedAt    time.Time  
    AccuracyMeters float64  
    IsMockLocation bool  
}
```

9.3 Route

```
type Route struct {  
    WalkID WalkID  
    Points []LocationPoint  
}
```

9.4 Domain Kararı

Route, Walk aggregate'inin çalışma girdisidir ancak binlerce GPS noktası doğrudan Walk nesnesinin sürekli yüklenen parçası olmamalıdır.

Application katmanı gerektiğinde route'u ayrı kaynaktan okuyup engine input'una dönüştürür.

9.5 Ölçek Notları

Raw GPS noktaları:

- Çok yüksek hacimli veri oluşturur.
- Profil ve Walk listeleme sorgularında gerekli değildir.
- Uzun süre sınırsız saklanmamalıdır.
- Ayrı saklama ve retention politikası gerektirir.
- Batch olarak alınmalı ve yazılmalıdır.

Domain açısından raw route authoritative yürüyüş girdisidir.

Ancak sürekli online sorgulanacak ana oyun state'i değildir.

10. TerritoryHex

10.1 Amaç

Oyundaki tek bir H3 bölgesinin aktif sahiplik ve dominance durumunu temsil eder.

MVP'nin en kritik aggregate root'udur.

10.2 Model

```
type TerritoryHex struct {  
    ID                HexID  
    CityID            CityID  
  
    OwnerID           *PlayerID  
    Dominance          int  
  
    LastUpdatedAt     time.Time  
    LastVisitedAt     *time.Time  
  
    Version            int64  
}
```

10.3 HexID

H3 cell ID domain içinde özel tip olmalıdır:

```
type HexID uint64
```

veya H3 kütüphanesinin kullanımına göre:

```
type HexID string
```

Domain, üçüncü taraf H3 tipini doğrudan dışarı taşımamalıdır.

Kesin storage tipi Database Design aşamasında belirlenecektir.

10.4 OwnerID

```
OwnerID = null
```

ise hex sahipsizdir.

```
OwnerID != null
```

ise bir oyuncu tarafından kontrol edilmektedir.

10.5 Dominance

Stored dominance database'de saklanan son uygulanmış değerdir.

Effective dominance engine tarafından hesaplanır:

```
Effective Dominance
=
Stored Dominance
-
Geçen günlere ait decay
```

Motor hesap sonunda yeni stored dominance üretir.

10.6 Version

Version eşzamanlı saldırıların çözümü için kullanılır.

Örnek:

```
Player A, Hex X version 12 üzerinden saldırır.
Player B, Hex X version 12 üzerinden saldırır.

İlk update:
version 12 → 13

İkinci update:
expected version 12 bulunamaz.
Conflict oluşur.
```

Application katmanı state'i yeniden okuyup motor hesabını tekrar çalıştırabilir.

10.7 Aggregate Sınırı

Her TerritoryHex bağımsız aggregate root'tur.

Bu tercih şu avantajları sağlar:

- Aynı şehirde farklı hex'ler paralel güncellenebilir.
- Bütün mahalle kilitlenmez.
- Conflict yalnızca aynı hex üzerinde oluşur.
- Yatay ölçekleme kolaylaşır.

11. HexVisit

11.1 Amaç

Bir yürüyüşün belirli bir hex üzerindeki doğrulanmış hareket özetini temsil eder.

Bu bir ana kalıcı aggregate değildir.

Game Engine'in ürettiği ara domain value object'tir.

11.2 Model

```
type HexVisit struct {  
    WalkID          WalkID  
    HexID           HexID  
    PresenceSeconds int  
    DistanceMeters  float64  
    EntryCount      int  
    FirstEnteredAt  time.Time  
    LastExitedAt    time.Time  
}
```

11.3 Kullanımı

HexVisit şu kararlar için kullanılır:

- Hex ziyareti geçerli mi?
- Empty capture yapılabilir mi?
- Owner defense uygulanabilir mi?
- Enemy attack uygulanabilir mi?

HexVisit aynı yürüyüşte aynı hex için tek bir özet kayıt olmalıdır.

12. HexChange

12.1 Amaç

Game Engine'in bir hex için ne olması gerektiğini belirten sonuç nesnesidir.

HexChange doğrudan database satırı değildir.

12.2 Model

```
type HexChange struct {  
    HexID HexID  
  
    ExpectedVersion int64  
  
    PreviousOwnerID *PlayerID  
    NewOwnerID      *PlayerID  
  
    PreviousDominance int  
    EffectiveDominance int  
    NewDominance      int  
  
    ChangeType HexChangeType  
    Reason      HexChangeReason  
  
    WalkID WalkID  
    OccurredAt time.Time  
}
```

12.3 ChangeType

```
type HexChangeType string  
  
const (  
    HexNoChange      HexChangeType = "NO_CHANGE"  
    HexCapturedEmpty HexChangeType = "EMPTY_CAPTURE"  
    HexDefended       HexChangeType = "OWNER_DEFENSE"  
    HexAttacked        HexChangeType = "ENEMY_ATTACK"  
    HexOwnershipChanged HexChangeType = "OWNERSHIP_TRANSFER"  
)
```

12.4 ExpectedVersion

Persistence katmanı değişikliği ancak mevcut version beklenen version ile eşleşiyorsa uygular.

Bu, yüksek eşzamanlılık için temel domain bilgisidir.

13. Territory Change Set

Bir yürüyüş birden fazla hex etkileyebilir.

Bütün değişiklikler şu sonuç altında gruplanır:

```
type TerritoryChangeSet struct {  
    WalkID    WalkID  
    Changes   []HexChange  
}
```

Ancak bütün hex'lerin tek aggregate gibi kilitlenmesi gerekmez.

Application katmanı transaction stratejisine göre değişiklikleri uygular.

13.1 Atomiklik Kararı

MVP için önerilen davranış:

Bir Walk sonucu, etkilediği bütün hex'ler ve skor değişiklikleri bakımından mümkün olduğunca tek transaction içinde uygulanmalıdır.

Ancak yüzlerce hex içeren anormal yürüyüşlerin transaction süresini uzatmaması gerekir.

Bu nedenle Calculator Rules tarafından yürüyüş başına makul bir maksimum etkilenmiş hex sınırı ilerde belirlenebilir.

MVP normal yürüyüş hedefi:

15-25 hex

Bu sayı tek transaction için yönetilebilir düzeydedir.

14. PlayerScore

14.1 Amaç

Oyuncunun haftalık ve lifetime ilerlemesini temsil eder.

14.2 Model

```
type PlayerScore struct {  
    PlayerID PlayerID  
    SeasonID SeasonID  
  
    WeeklyActiveHexes int  
    WeeklyCaptures    int  
    WeeklySteals        int  
    WeeklyDefenses      int
```

```
LifetimeCaptures    int64
LifetimeSteals        int64
LifetimeWalks         int64
LifetimeDistanceM     int64

Version              int64
UpdatedAt            time.Time
}
```

14.3 Aggregate Kararı

PlayerScore, Player aggregate'inin içine gömülmez.

Ayrı aggregate olarak tutulur.

Neden:

- Player profili sık okunabilir.
- Score sık güncellenebilir.
- Score güncellemesi profil kaydını kilitlememelidir.
- Leaderboard sorguları profil modelinden bağımsız olmalıdır.

14.4 Active Hex Count

`WeeklyActiveHexes` denormalize edilmiş bir sayaç olabilir.

Authoritative ownership bilgisi TerritoryHex kayıtlarındadır.

Sayaçların sapması durumunda yeniden hesaplanabilir olması gerekir.

Bu nedenle:

```
TerritoryHex ownership = source of truth
PlayerScore counters   = optimized derived state
```

14.5 Eşzamanlılık

PlayerScore da version alanına sahip olmalıdır.

Ancak aynı oyuncunun aynı anda birden fazla Walk sonucu işlenirse skor güncellemeleri conflict oluşturabilir.

Database tasarımında şu seçenekler değerlendirilecektir:

- Atomic increment
- Optimistic locking
- Serialized per-player processing

- Event tabanlı score projection

MVP için atomic increment öncelikli adaydır.

15. ScoreChange

15.1 Amaç

Engine'in skor üzerinde yapılması gereken değişiklikleri belirtir.

15.2 Model

```
type ScoreChange struct {  
    PlayerID PlayerID  
    SeasonID SeasonID  
    Metric    ScoreMetric  
    Delta     int64  
    Reason    ScoreChangeReason  
    WalkID    WalkID  
}
```

Örnek:

```
EMPTY_CAPTURE  
→ WeeklyActiveHexes +1  
→ WeeklyCaptures +1  
→ LifetimeCaptures +1
```

Ownership transfer:

```
Attacker WeeklyActiveHexes +1  
Defender WeeklyActiveHexes -1  
Attacker WeeklySteals +1  
Attacker LifetimeSteals +1
```

Skor değişiklikleri doğrudan yeni toplam değer üretmek yerine delta olarak üretilebilir.

Bu yaklaşım eşzamanlı increment işlemlerine daha uygundur.

16. Season

16.1 Amaç

Haftalık rekabet dönemini temsil eder.

16.2 Model

```
type Season struct {
    ID          SeasonID
    Number      int
    Status       SeasonStatus
    StartsAt    time.Time
    EndsAt      time.Time
    RuleSetVersion RuleSetVersion
}
```

16.3 SeasonStatus

```
type SeasonStatus string

const (
    SeasonScheduled SeasonStatus = "SCHEDULED"
    SeasonActive     SeasonStatus = "ACTIVE"
    SeasonClosed     SeasonStatus = "CLOSED"
    SeasonArchived   SeasonStatus = "ARCHIVED"
)
```

16.4 Aggregate Kararı

Season bütün oyuncu skorlarını içinde tutmaz.

Yanlış:

```
type Season struct {
    Scores []PlayerScore
}
```

Doğru:

PlayerScore, SeasonID ile Season'a bağlanır.

16.5 Reset

Weekly reset sırasında:

- Yeni Season oluşturulur.
- Yeni haftalık score state'i başlatılır.
- Lifetime değerler korunur.
- Territory ownership doğrudan silinmez.

Season kapanışı bütün oyuncuları aynı transaction içinde güncellememelidir.

Yeni sezon geçişi tarihsel SeasonID üzerinden yönetilmelidir.

17. RuleSet

17.1 Amaç

Motorun değiştirilebilir iş parametrelerini temsil eder.

17.2 Model

```
type RuleSet struct {
    Version    RuleSetVersion
    Status     RuleSetStatus

    Walk       WalkRules
    Movement   MovementRules
    Territory   TerritoryRules
    Season     SeasonRules

    CreatedAt  time.Time
    ActivatedAt *time.Time
}
```

17.3 RuleSetStatus

```
type RuleSetStatus string

const (
    RuleSetDraft    RuleSetStatus = "DRAFT"
    RuleSetActive   RuleSetStatus = "ACTIVE"
    RuleSetRetired  RuleSetStatus = "RETIRED"
)
```

17.4 Değişmezlik

Bir RuleSet aktif olduktan ve yürüyüşlerde kullanıldıktan sonra içerik olarak değiştirilmemelidir.

Yanlış:

```
mvp-0.1.0 içindeki attack damage 15'ten 12'ye çevrilir.
```

Doğru:

```
mvp-0.1.0 → retired  
mvp-0.2.0 → active
```

Bu sayede replay yapılabilir.

17.5 Şehir İlişkisi

MVP'de global RuleSet yeterlidir.

City içinde:

```
ActiveRuleSetVersion
```

referansı tutulabilir.

Gelecekte şehir bazlı farklı RuleSet mümkün olur.

18. DomainEvent

18.1 Amaç

Motor sonucunda gerçekleşen domain olayını temsil eder.

18.2 Model

```
type DomainEvent struct {  
    ID          EventID  
    Type        EventType  
    AggregateType AggregateType  
    AggregateID string  
  
    WalkID      *WalkID  
    PlayerID     *PlayerID  
}
```

```
HexID      *HexID

OccurredAt time.Time
Payload    EventPayload
}
```

18.3 Event Türleri

```
WALK_VALIDATED
WALK_PARTIALLY_VALIDATED
WALK_REJECTED

EMPTY_HEX_CAPTURED
HEX_DEFENDED
HEX_ATTACKED
HEX_UNDER_THREAT
HEX_CAPTURED_FROM_RIVAL
HEX_LOST

PLAYER_SCORE_CHANGED
SEASON_STARTED
SEASON_ENDED
PLAYER_BECAME_ALPHA
```

18.4 Domain Event ve Outbox Ayrımı

DomainEvent iş kavramıdır.

OutboxRecord database/infrastructure kavramıdır.

```
DomainEvent
→ Application mapper
→ OutboxRecord
```

Motor outbox tablosunu bilmez.

19. Leaderboard Read Model

Leaderboard authoritative aggregate değildir.

Oyuncuların sırası şu kaynaklardan türetilir:

```
PlayerScore
Season
Player profile summary
```

Örnek read model:

```
type LeaderboardEntry struct {
    Rank          int
    PlayerID      PlayerID
    DisplayName    string
    ActiveHexes    int
    WeeklySteals   int
    WeeklyDefenses int
}
```

19.1 Ölçek Kararı

Leaderboard:

- Her request'te bütün TerritoryHex kayıtlarından hesaplanmamalıdır.
- PlayerScore üzerinden oluşturulmalıdır.
- Cache veya projection ile sunulabilir.
- Birkaç saniyelik eventual consistency kabul edilebilir.

Ana oyun işlemi leaderboard güncellemesini beklememelidir.

20. Map Territory Read Model

Mobil harita açıldığında her TerritoryHex aggregate'inin bütün detaylarının gönderilmesi gerekmez.

Harita için ayrı bir özet model kullanılır:

```
type TerritoryMapCell struct {
    HexID      HexID
    OwnerID    *PlayerID
    DominanceBand DominanceBand
    State      TerritoryDisplayState
}
```

20.1 DominanceBand

Mobil istemciye her zaman kesin dominance değeri göndermek zorunlu olmayabilir.

Örnek:

LOW
MEDIUM
HIGH
CRITICAL

MVP’de kesin deęer de gnderilebilir; bu API tasarımında kararlařtırılacaktır.

20.2 lek Notu

Harita verileri:

- Bounding box
- H3 parent region
- Tile
- Viewport

bazlı istenmelidir.

Bütün řehirdeki hex’ler tek API response iinde gnderilmemelidir.

Bu karar API Design ařamasında uygulanacaktır.

21. Walk Processing Result

Application katmanında bir Walk iřleminin btn sonucunu temsil eder:

```
type WalkProcessingResult struct {  
    WalkID          WalkID  
    Status          WalkStatus  
    Validation      WalkValidationResult  
    Metrics         WalkMetrics  
    TerritoryChanges []HexChange  
    ScoreChanges    []ScoreChange  
    Events          []DomainEvent  
    Metadata        CalculationMetadata  
}
```

Bu nesne:

- Motor sonucunu tařır.
- Persistence deęildir.
- API response deęildir.
- Application iřlem sonucudur.

22. Domain İlişki Özeti

```
Player
├── 1:N Pet
├── 1:N Walk
├── 1:N PlayerScore
└── 1:N TerritoryHex ownership

City
├── 1:N Walk
├── 1:N TerritoryHex
└── N:1 Active RuleSet

Walk
├── N:1 Player
├── N:1 Pet
├── N:1 City
├── 1:N LocationPoint
├── 1:N HexChange
└── 1:N DomainEvent

TerritoryHex
├── N:1 City
└── N:0..1 Owner Player

PlayerScore
├── N:1 Player
└── N:1 Season

Season
└── N:1 RuleSet
```

23. Aggregate Root'lar

MVP aggregate root listesi:

```
Player
Pet
Walk
TerritoryHex
PlayerScore
Season
RuleSet
```

DomainEvent aggregate değildir.

HexChange value/result object'tir.

LocationPoint yürüyüş girdisidir.

Leaderboard ve MapCell read model'dir.

24. Transaction Sınırı

Bir Walk başarıyla çözümlendiğinde application katmanı şu authoritative değişiklikleri uygulamalıdır:

1. Walk state
2. Etkilenen TerritoryHex state'leri
3. PlayerScore delta'ları
4. Domain event outbox kayıtları

İdeal olarak aynı transaction içinde:

Walk → RESOLVED
Hex changes → applied
Score changes → applied
Events → outbox inserted

Bu, motor sonucunun yarım uygulanmasını engeller.

24.1 Transaction İçine Girmemesi Gereken İşler

Push notification gönderme
Leaderboard cache güncelleme
Analytics gönderme
WebSocket broadcast
Harita cache temizleme
Email gönderme

Bunlar transaction sonrasında event/outbox üzerinden yapılmalıdır.

25. Eşzamanlı Hex Güncellemeleri

En kritik yarış durumu:

İki Walk aynı hex'in aynı version'ı üzerinden hesaplanır.

HexChange şu bilgileri taşır:

```
HexID
ExpectedVersion
PreviousOwner
PreviousDominance
NewOwner
NewDominance
```

Update koşulu:

```
Update only if current_version = expected_version
```

Başarısızsa:

```
CONFLICT
```

Application:

1. Hex'in yeni state'ini okur.
2. Gerekli kısmı yeniden hesaplar.
3. Sınırlı sayıda retry uygular.

Sınırsız retry yapılmaz.

26. Aynı Oyuncunun Paralel Walk İşlemleri

Bir oyuncunun iki aktif yürüyüş başlatması engellenebilir.

MVP domain kuralı:

```
Bir oyuncunun aynı anda en fazla bir ACTIVE Walk'ı olabilir.
```

Bu kural:

- Farming'i azaltır.
- State yönetimini kolaylaştırır.
- Yanlışlıkla çift yürüyüş başlatılmasını önler.

Ancak yürüyüşlerin backend processing aşaması kısa süreli olarak paralel çalışabilir.

PlayerScore güncellemeleri atomic delta veya version kontrolüyle korunmalıdır.

27. Veri Sahipliği ve Source of Truth

Veri	Source of Truth
Walk lifecycle	Walk
Raw GPS route	Walk route storage
Hex sahibi	TerritoryHex
Stored dominance	TerritoryHex
Effective dominance	Game Engine hesabı
Weekly counters	PlayerScore
Lifetime counters	PlayerScore
Leaderboard rank	Read model / projection
Harita görünümü	Territory read model
Aktif kurallar	RuleSet
Sezon zamanı	Season
Notification durumu	Infrastructure
Event delivery	Outbox / messaging

28. Kalıcı ve Hesaplanabilir Veriler

Kalıcı Tutulması Gerekenler

Player
Pet
City
Walk metadata
Raw veya sıkıştırılmış route
TerritoryHex ownership
Stored dominance
TerritoryHex version
PlayerScore counters
Season
RuleSet versions
Outbox records

Hesaplanabilir Veriler

Effective dominance
Walk sırasında visited hex listesi
Harita görünüm bandı
Leaderboard rank
Oyuncunun aktif hex listesi
Mahalle özeti

Bazı hesaplanabilir veriler performans için cache veya projection olarak saklanabilir.

Ancak authoritative kaynakları belli olmalıdır.

29. Domain Invariant'ları

Sistem boyunca korunması gereken temel kurallar:

1. Dominance 0 ile MaxDominance arasında olmalı.
2. OwnerID null ise owned dominance anlamı taşımamalı.
3. Bir Walk yalnızca bir kez resolved olabilir.
4. Aynı Walk iki kez score değişikliği üretmemeli.
5. Aynı Player aynı anda en fazla bir ACTIVE Walk'a sahip olmalı.
6. RuleSetVersion boş olamaz.
7. Kullanılmış RuleSet değiştirilemez.
8. TerritoryHex version her state değişiminde artmalı.
9. Ownership transfer yalnızca dominance sıfıra düştüğünde oluşmalı.
10. Passive decay doğrudan owner değiştirmemeli.
11. PlayerScore weekly değerleri bir Season'a bağlı olmalı.
12. Event occurred time motorun EvaluatedAt değerinden gelmeli.

30. Yüksek Trafik İçin Tasarım Kararları

30.1 Walk Upload

Raw GPS noktaları tek tek endpoint'e sürekli yazılmak yerine:

- Küçük batch'ler halinde
- Sıralı sequence numarasıyla
- Idempotent şekilde

gönderilebilir.

Kesin API kararı API Design aşamasında verilecektir.

30.2 Territory Writes

Her Walk yalnızca dokunduğu hex'leri günceller.

Bütün şehir state'i okunmaz veya yazılmaz.

30.3 Map Reads

Harita verisi viewport bazlı okunur.

Bütün şehir response olarak dönmez.

30.4 Leaderboard Reads

PlayerScore read model veya cache üzerinden okunur.

TerritoryHex üzerinde pahalı aggregate sorgusu her istekte yapılmaz.

30.5 Event Processing

Notification ve analytics ana Walk transaction'ını bloklamaz.

Outbox üzerinden asenkron işlenir.

30.6 RuleSet Cache

Aktif RuleSet her Walk için database'den tekrar okunmak zorunda değildir.

Application seviyesinde immutable version cache kullanılabilir.

Motor cache bilmez.

31. MVP'de Kapsam Dışı Domain Nesneleri

Aşağıdaki modeller şimdilik eklenmeyecektir:

```
Guild
GuildMember
ProfessionalWalkerProfile
BreedClass
Booster
Inventory
CoinWallet
SponsorTerritory
EventBattle
```

NPCPlayer
NPCBehavior
Achievement
Badge inventory
Chat
SocialFeed

Bu nesneler ileride ihtiyaç olduğunda ayrı domain modülleri olarak eklenebilir.

32. Database Design'e Aktarılabacak Kritik Kararlar

Database aşamasında şu konular kesinleştirilecektir:

1. PostgreSQL ve PostGIS kullanımı
2. H3 ID'nin bigint mi text mi saklanacağı
3. Raw GPS point saklama ve partition stratejisi
4. TerritoryHex unique key yapısı
5. Optimistic locking update sorgusu
6. PlayerScore atomic increment yaklaşımı
7. Walk idempotency constraint'leri
8. RuleSet JSONB ve normalize alan dengesi
9. Outbox tablosu
10. Season ve weekly score ayrımı
11. Index stratejisi
12. Harita viewport sorguları
13. Veri retention politikası
14. Yüksek hacimli tablolar için partition gereksinimi
15. Read replica ve cache ihtiyacının ne zaman başlayacağı

33. API Design'e Aktarılabacak Kritik Kararlar

API aşamasında:

1. Walk start endpoint
2. GPS batch upload
3. Walk finish ve resolve akışı
4. Idempotency key
5. Processing status
6. Viewport bazlı territory endpoint
7. Profile summary endpoint
8. Leaderboard pagination
9. RuleSet'in mobile'a ne kadarının gönderileceği
10. Rate limiting
11. Payload compression

- 12. Retry davranışı
- 13. Offline walk senkronizasyonu

belirlenecektir.

34. Definition of Done

MVP Domain Model aşağıdaki koşulları sağladığında uygulanabilir kabul edilir:

1. Walk bağımsız aggregate olarak tanımlı.
2. TerritoryHex bağımsız aggregate root.
3. PlayerScore, Player profilinden ayrılmış.
4. Raw GPS verisi ana Player veya Walk sorgusunu şişirmiyor.
5. HexChange ExpectedVersion taşıyor.
6. RuleSet immutable ve versiyonlu.
7. Domain Event ile Outbox ayrılmış.
8. Leaderboard authoritative aggregate değil.
9. Map data ayrı read model olarak ele alınmış.
10. Aynı Walk'ın iki kez işlenmesi engellenebilir.
11. Aynı hex üzerindeki paralel güncellemeler tespit edilebilir.
12. Domain nesneleri database modellerine bağımlı değil.
13. Motor input ve output modelleri domain kavramlarıyla uyumlu.
14. Binlerce kullanıcının farklı hex'leri paralel güncelleyebilmesine izin veriyor.
15. Aynı hot hex üzerindeki conflict yalnızca ilgili hex'i etkiliyor.

35. Nihai Özet

MVP domain modeli üç ana authoritative state etrafında kurulmuştur:

Walk
TerritoryHex
PlayerScore

Bunları destekleyen temel yapılar:

Player
Pet
City
Season
RuleSet
DomainEvent

Yüksek trafik açısından en önemli kararlar:

Her hex bağımsız aggregate root.
Bütün şehir tek aggregate değil.
Score, Player profilinden ayrı.
Leaderboard ve harita ayrı read model.
Raw GPS verisi sıcak domain state'inden ayrı.
Hex güncellemeleri version kontrollü.
Walk işlemleri idempotent.
Yan etkiler outbox üzerinden ayrıştırılmış.

Bu model sayesinde aynı anda binlerce kullanıcı farklı yürüyüşler gönderebilir; yalnızca aynı hex'e aynı anda yapılan işlemler conflict üretir.

Sıradaki aşama:

Pet Territory Wars — MVP Database Design v1.0